

C++
11

可以采取另外一种方法, 通过使用标准库功能 `mem_fn` 来让编译器负责推断成员的类型。和 `function` 一样, `mem_fn` 也定义在 `functional` 头文件中, 并且可以从成员指针生成一个可调用对象; 和 `function` 不同的是, `mem_fn` 可以根据成员指针的类型推断可调用对象的类型, 而无须用户显式地指定:

```
find_if(svec.begin(), svec.end(), mem_fn(&string::empty));
```

我们使用 `mem_fn(&string::empty)` 生成一个可调用对象, 该对象接受一个 `string` 实参, 返回一个 `bool` 值。

`mem_fn` 生成的可调用对象可以通过对象调用, 也可以通过指针调用:

```
auto f = mem_fn(&string::empty); // f 接受一个 string 或者一个 string*
f(*svec.begin());               // 正确: 传入一个 string 对象, f 使用.*调用 empty
f(&svec[0]);                     // 正确: 传入一个 string 的指针, f 使用->*调用 empty
```

实际上, 我们可以认为 `mem_fn` 生成的可调用对象含有一对重载的函数调用运算符: 一个接受 `string*`, 另一个接受 `string&`。

使用 bind 生成一个可调用对象

出于完整性的考虑, 我们还可以使用 `bind` (参见 10.3.4 节, 第 354 页) 从成员函数生成一个可调用对象:

```
// 选择范围内的每个 string, 并将其 bind 到 empty 的第一个隐式实参上
auto it = find_if(svec.begin(), svec.end(),
                  bind(&string::empty, _1));
```

和 `function` 类似的地方是, 当我们使用 `bind` 时, 必须将函数中用于表示执行对象的隐式形参转换成显式的。和 `mem_fn` 类似的地方是, `bind` 生成的可调用对象的第一个实参既可以是 `string` 的指针, 也可以是 `string` 的引用:

```
auto f = bind(&string::empty, _1);
f(*svec.begin()); // 正确: 实参是一个 string, f 使用.*调用 empty
f(&svec[0]);       // 正确: 实参是一个 string 的指针, f 使用->*调用 empty
```

19.4.3 节练习

练习 19.18: 编写一个函数, 使用 `count_if` 统计在给定的 `vector` 中有多少个空 `string`。

练习 19.19: 编写一个函数, 令其接受 `vector<Sales_data>` 并查找平均价格高于某个值的第一个元素。

19.5 嵌套类

一个类可以定义在另一个类的内部, 前者称为**嵌套类** (`nested class`) 或**嵌套类型** (`nested type`)。嵌套类常用于定义作为实现部分的类, 比如我们在文本查询示例中使用的 `QueryResult` 类 (参见 12.3 节, 第 430 页)。

844

嵌套类是一个独立的类, 与外层类基本没什么关系。特别是, 外层类的对象和嵌套类的对象是相互独立的。在嵌套类的对象中不包含任何外层类定义的成员; 类似的, 在外层类的对象中也不包含任何嵌套类定义的成员。

嵌套类的名字在外层类作用域中是可见的，在外层类作用域之外不可见。和其他嵌套的名字一样，嵌套类的名字不会和别的作用域中的同一个名字冲突。

嵌套类中成员的种类与非嵌套类是一样的。和其他类类似，嵌套类也使用访问限定符来控制外界对其成员的访问权限。外层类对嵌套类的成员没有特殊的访问权限，同样，嵌套类对外层类的成员也没有特殊的访问权限。

嵌套类在其外层类中定义了一个类型成员。和其他成员类似，该类型的访问权限由外层类决定。位于外层类 `public` 部分的嵌套类实际上定义了一种可以随处访问的类型；位于外层类 `protected` 部分的嵌套类定义的类型只能被外层类及其友元和派生类访问；位于外层类 `private` 部分的嵌套类定义的类型只能被外层类的成员和友元访问。

声明一个嵌套类

我们为 12.3.2 节（第 432 页）的 `TextQuery` 类定义了一个名为 `QueryResult` 的嵌套类，这两个类密切相关。`QueryResult` 类的主要作用是表示 `TextQuery` 对象上 `query` 操作的结果，显然将 `QueryResult` 用作其他目的没有任何意义。为了充分体现这种紧密的相关性，我们可以把 `QueryResult` 定义成 `TextQuery` 的成员。

```
class TextQuery {
public:
    class QueryResult;           // 嵌套类稍后定义
    // 其他成员与 12.3.2 节（第 432 页）一致
};
```

我们只需对原来的 `TextQuery` 类做一处改动，即将 `QueryResult` 声明成嵌套类。因为 `QueryResult` 是一个类型成员（参见 7.4.1 节，第 254 页），所以必须对它先声明后使用，尤其是必须先声明 `QueryResult`，再将它作为 `query` 成员的返回类型。类的其他成员没有任何变化。

在外层类之外定义一个嵌套类

◀ 845

我们在 `TextQuery` 内声明了 `QueryResult`，但是没有给出它的定义。和成员函数一样，嵌套类必须声明在类的内部，但是可以定义在类的内部或者外部。

当我们在外层类之外定义一个嵌套类时，必须以外层类的名字限定嵌套类的名字：

```
// QueryResult 是 TextQuery 的成员，下面的代码负责定义 QueryResult
class TextQuery::QueryResult {
    // 位于类的作用域内，因此我们不必对 QueryResult 形参进行限定
    friend std::ostream&
        print(std::ostream&, const QueryResult&);
public:
    // 无须定义 QueryResult::line_no
    // 嵌套类可以直接使用外层类的成员，无须对该成员的名字进行限定
    QueryResult(std::string,
                std::shared_ptr<std::set<line_no>>,
                std::shared_ptr<std::vector<std::string>>);
    // 其他成员与 12.3.2 节（第 432 页）一致
};
```

和原来的类相比唯一的改动是，我们无须在 `QueryResult` 内定义 `line_no` 成员了。因为该成员属于 `TextQuery`，所以 `QueryResult` 可以直接访问它而不必再定义一次。



在嵌套类在其外层类之外完成真正的定义之前，它都是一个不完全类型（参见 7.3.3 节，第 250 页）。

定义嵌套类的成员

在这个版本的 `QueryResult` 类中，我们并没有在类的内部定义其构造函数。要想为其定义构造函数，必须指明 `QueryResult` 是嵌套在 `TextQuery` 的作用域之内的。具体做法是使用外层类的名字限定嵌套类的名字：

```
// QueryResult 类嵌套在 TextQuery 类中
// 下面的代码为 QueryResult 类定义名为 QueryResult 的成员
TextQuery::QueryResult::QueryResult(string s,
                                     shared_ptr<set<line_no>> p,
                                     shared_ptr<vector<string>> f):
    sought(s), lines(p), file(f) { }
```

从右向左阅读函数的名字可知我们定义的是 `QueryResult` 类的构造函数，而 `QueryResult` 类是嵌套在 `TextQuery` 类中的。该构造函数除了把实参值赋给对应的数据成员之外，没有做其他工作。

嵌套类的静态成员定义

如果 `QueryResult` 声明了一个静态成员，则该成员的定义将位于 `TextQuery` 的作用域之外。例如，假设 `QueryResult` 有一个静态成员，则该成员的定义将形如：

```
// QueryResult 类嵌套在 TextQuery 类中，
// 下面的代码为 QueryResult 定义一个静态成员
int TextQuery::QueryResult::static_mem = 1024;
```

嵌套类作用域中的名字查找

名字查找的一般规则（参见 7.4.1 节，第 254 页）在嵌套类中同样适用。当然，因为嵌套类本身是一个嵌套作用域，所以还必须查找嵌套类的外层作用域。这种作用域嵌套的性质正好可以说明为什么我们不在 `QueryResult` 的嵌套版本中定义 `line_no`。原来的 `QueryResult` 类定义了该成员，从而使其成员可以避免使用 `TextQuery::line_no` 的形式。然而 `QueryResult` 的嵌套类版本本身就是定义在 `TextQuery` 中的，所以我们不需要再使用 `typedef`。嵌套的 `QueryResult` 无须说明 `line_no` 属于 `TextQuery` 就可以直接使用它。

如我们所知，嵌套类是其外层类的一个类型成员，因此外层类的成员可以像使用任何其他类型成员一样使用嵌套类的名字。因为 `QueryResult` 嵌套在 `TextQuery` 中，所以 `TextQuery` 的 `query` 成员可以直接使用名字 `QueryResult`：

```
// 返回类型必须指明 QueryResult 是一个嵌套类
TextQuery::QueryResult
TextQuery::query(const string &sought) const
{
    // 如果我们没有找到 sought，则返回 set 的指针
    static shared_ptr<set<line_no>> nodata(new set<line_no>);
    // 使用 find 而非下标以避免向 wm 中添加单词
    auto loc = wm.find(sought);
    if (loc == wm.end())
```

```
        return QueryResult(sought, nodata, file);           // 没有找到
    else
        return QueryResult(sought, loc->second, file);
}
```

和过去一样，返回类型不在类的作用域中（参见 7.4 节，第 253 页），因此我们必须指明函数的返回值是 `TextQuery::QueryResult` 类型。不过在函数体内部我们可以直接访问 `QueryResult`，比如上面的 `return` 语句就是这样。

嵌套类和外层类是相互独立的

尽管嵌套类定义在其外层类的作用域中，但是读者必须谨记外层类的对象和嵌套类的对象没有任何关系。嵌套类的对象只包含嵌套类定义的成员；同样，外层类的对象只包含外层类定义的成员，在外层类对象中不会有任何嵌套类的成员。

说得再具体一些，`TextQuery::query` 的第二条 `return` 语句

```
return QueryResult(sought, loc->second, file);
```

使用了 `TextQuery` 对象的数据成员，而 `query` 正是用它们来初始化 `QueryResult` 对象的。因为在一个 `QueryResult` 对象中不包含其外层类的成员，所以必须使用上述成员构造我们返回的 `QueryResult` 对象。

◀ 847

19.5 节练习

练习 19.20：将你的 `QueryResult` 类嵌套在 `TextQuery` 中，然后重新运行 12.3.2 节（第 435 页）中使用了 `TextQuery` 的程序。

19.6 union：一种节省空间的类

联合（union）是一种特殊的类。一个 union 可以有多个数据成员，但是在任意时刻只有一个数据成员可以有值。当我们给 union 的某个成员赋值之后，该 union 的其他成员就变成未定义的状态了。分配给一个 union 对象的存储空间至少要能容纳它的最大的数据成员。和其他类一样，一个 union 定义了一种新类型。

类的某些特性对 union 同样适用，但并非所有特性都如此。union 不能含有引用类型的成员，除此之外，它的成员可以是绝大多数类型。在 C++11 新标准中，含有构造函数或析构函数的类类型也可以作为 union 的成员类型。union 可以为其成员指定 `public`、`protected` 和 `private` 等保护标记。默认情况下，union 的成员都是公有的，这一点与 `struct` 相同。

union 可以定义包括构造函数和析构函数在内的成员函数。但是由于 union 既不能继承自其他类，也不能作为基类使用，所以在 union 中不能含有虚函数。

定义 union

union 提供了一种有效的途径使得我们可以方便地表示一组类型不同的互斥值。举个例子，假设我们需要处理一些不同种类的数字数据和字符数据，则在此过程中可以定义一个 union 来保存这些值：

```
// Token 类型的对象只有一个成员，该成员的类型可能是下列类型中的任意一种
union Token {
```